

# Documento de Diseño

## Laboratorio 3: Lógica Secuencial y Controladores

Grupo: Emmanuel Rojas Fernandez  
Gabriel Fernandez Vargas

27 de septiembre de 2025

### Resumen

Este documento presenta dos propuestas de diseño para el juego *Memoria* implementado sobre una interfaz VGA de  $640 \times 480$ : (A) una FSM monolítica y (B) un diseño modular con FSM maestro y sub-FSMs. Para cada propuesta se incluyen: diagrama de bloques, diagrama de estados (FSM), tabla de transiciones, señales de entrada/salida, consideraciones temporales (contador de 15 s), método anti-rebote, LFSR para selección aleatoria, y recomendaciones para testbench e integración en FPGA. Finalmente se da la justificación de la propuesta elegida.

# Índice

|                                                      |          |
|------------------------------------------------------|----------|
| <b>1. Introducción</b>                               | <b>3</b> |
| <b>2. Arquitectura general (componentes comunes)</b> | <b>3</b> |
| <b>3. Propuesta A — FSM Monolítica</b>               | <b>3</b> |
| 3.1. Descripción general . . . . .                   | 3        |
| 3.2. Estados . . . . .                               | 3        |
| 3.3. Tabla de transiciones . . . . .                 | 4        |
| 3.4. Entradas / salidas . . . . .                    | 4        |
| 3.5. Otros aspectos . . . . .                        | 4        |
| 3.6. Testbench . . . . .                             | 5        |
| <b>4. Propuesta B — Diseño Modular</b>               | <b>6</b> |
| 4.1. Descripción general . . . . .                   | 6        |
| 4.2. Submódulos principales . . . . .                | 6        |
| 4.3. FSM maestro . . . . .                           | 6        |
| 4.4. Ventajas . . . . .                              | 6        |
| 4.5. Testbenches . . . . .                           | 7        |

# 1. Introducción

Este laboratorio requiere diseñar un controlador VGA y la lógica de control para un juego *Memoria* de 16 cartas (8 pares), entre dos jugadores, con turno de 15 s. Todas las señales de control deben provenir de FSM(s) y el sistema debe ser simulable y sintetizable en FPGA. Se proponen dos enfoques: uno simple (rápido de implementar) y otro modular (más mantenible y escalable).

## 2. Arquitectura general (componentes comunes)

Independiente de la propuesta elegida, los módulos mínimos son:

- **vga\_controller**: genera `hsync`, `vsync`, `pixel_x`, `pixel_y`, y señal `visible`.
- **card\_mem**: memoria que almacena el símbolo (índice) de cada posición (16 posiciones).
- **card\_state**: vector con estado de cada carta (HIDDEN, REVEALED, FOUND).
- **debounce**: módulo que filtra rebotes de botones.
- **timer**: contador parametrizable para 15 s (y versiones cortas para simulación).
- **lfsr**: generador pseudoaleatorio para selecciones en timeout.
- **fsm(s)**: controlador(es) del flujo del juego (propuesta A o B).
- **renderer**: módulo que traduce `card_mem` y `card_state` a colores/pixels.

## 3. Propuesta A — FSM Monolítica

### 3.1. Descripción general

La FSM monolítica contiene toda la lógica de control del juego en un único bloque: inicialización, espera de selección, mostrar cartas, comprobar coincidencia, actualizar puntajes, manejar timeout y terminar el juego. Es la opción más sencilla de entender y entregar rápidamente.

### 3.2. Estados

1. **RESET** – inicialización global.
2. **INIT\_BOARD** – barajar (`shuffle`) y cargar `card_mem`.
3. **WAIT\_PLAYER\_ACTION** – espera primera selección.
4. **SHOW\_FIRST** – revelar primera carta.
5. **WAIT\_SECOND\_ACTION** – espera segunda selección.

6. **SHOW\_SECOND** – revelar segunda carta.
7. **CHECK\_MATCH** – comparar símbolos.
8. **MATCH\_FOUND** – actualizar puntos, marcar cartas como FOUND.
9. **NO\_MATCH** – esperar un tiempo breve y ocultar cartas.
10. **AUTO\_SELECT** – selección aleatoria por timeout.
11. **CHECK\_END** – verificar si todas las parejas fueron encontradas.
12. **GAME\_OVER** – mostrar ganador, espera reset.

### 3.3. Tabla de transiciones

| Estado actual      | Condición      | Siguiente estado   |
|--------------------|----------------|--------------------|
| RESET              | power up / rst | INIT_BOARD         |
| INIT_BOARD         | shuffle done   | WAIT_PLAYER_ACTION |
| WAIT_PLAYER_ACTION | btn_pressed    | SHOW_FIRST         |
| SHOW_FIRST         | reveal_done    | WAIT_SECOND_ACTION |
| WAIT_SECOND_ACTION | btn_pressed    | SHOW_SECOND        |
| WAIT_SECOND_ACTION | timer_15s      | AUTO_SELECT        |
| SHOW_SECOND        | reveal_done    | CHECK_MATCH        |
| CHECK_MATCH        | match          | MATCH_FOUND        |
| CHECK_MATCH        | no_match       | NO_MATCH           |
| MATCH_FOUND        | update_done    | CHECK_END          |
| NO_MATCH           | hide_done      | CHECK_END          |
| CHECK_END          | all_found      | GAME_OVER          |
| CHECK_END          | else           | WAIT_PLAYER_ACTION |
| GAME_OVER          | reset          | RESET              |

Cuadro 1: Tabla de transiciones — Propuesta A

### 3.4. Entradas / salidas

**Entradas:** clk, rst\_n, btn\_select[15:0].

**Salidas:** reveal\_pos, score\_p1/p2, current\_player, señales VGA.

### 3.5. Otros aspectos

- **Temporizador:** 15 s con contador parametrizable.
- **Debounce:** FSM pequeña que valida pulsos estables.
- **LFSR:** Generador pseudoaleatorio para selección automática.

### 3.6. Testbench

El testbench debe validar:

- Reset, entradas de botones con rebote y sin rebote.
- Casos de timeout (selección automática).
- Actualización correcta de puntos tras coincidencias.

## 4. Propuesta B — Diseño Modular

### 4.1. Descripción general

Propuesta más limpia: varias FSM pequeñas y especializadas. Un FSM maestro (orquestador) gestiona turnos y eventos; sub-FSMs controlan tablero, entradas, temporizador y renderizado. Ventaja: modularidad, facilidad de pruebas y escalabilidad.

### 4.2. Submódulos principales

- **fsm\_master**: coordina turnos, recibe eventos y ordena acciones.
- **fsm\_board**: mantiene `card_state[15:0]` y responde a comandos.
- **fsm\_timer**: controla tiempo de turno.
- **fsm\_input**: filtra rebotes y genera eventos de selección.
- **lfsr**: pseudoaleatorio para auto-select.

### 4.3. FSM maestro

q1: inicialización del sistema  
q2: generación de semilla aleatoria  
q3: Barajar y cargar mazo  
q4: Pantalla de Inicio  
q5: preparación de turnos  
q6: Lectura y filtrado de la primera selección  
q7: Revelar primera carta  
q8: Pausa tras primera revelación  
q9: Lectura y filtrado de la segunda selección (debounce)  
q10: Revelar segunda carta  
q11: Evaluación de la pareja (comparador)  
q12: manejo de pareja encontrada  
q13: manejo de no coincidencia (mostrar y ocultar)  
q14: selección automática por timeout (auto pick)  
q15: Verificación de fin de juego  
q16: pantalla final/corre de partida

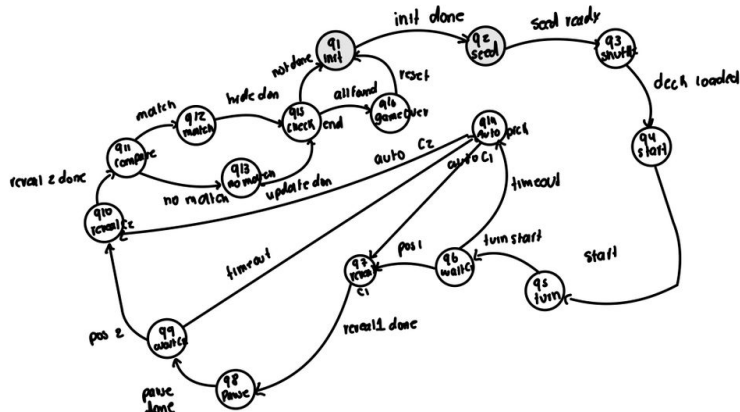


Figura 1: FSM maestro y comunicación con sub-FSMs (Propuesta B).

### 4.4. Ventajas

- Depuración modular.

- **Reutilización de componentes.**
- **Escalabilidad** para agregar nuevas funciones.

#### 4.5. Testbenches

1. **TB\_board:** prueba reveal/hide/found.
2. **TB\_timer:** prueba enable/disable y timeout.
3. **TB\_input:** prueba debounce.
4. **TB\_top:** simula partidas completas.

# Propuesta Elegida

Luego de analizar ambas propuestas, se eligió la **Propuesta B (FSM modular/jerárquica)** como la mejor alternativa.

## Justificación

1. Claridad y organización.
2. Facilidad de depuración.
3. Escalabilidad.
4. Buenas prácticas de ingeniería.

## FSM final

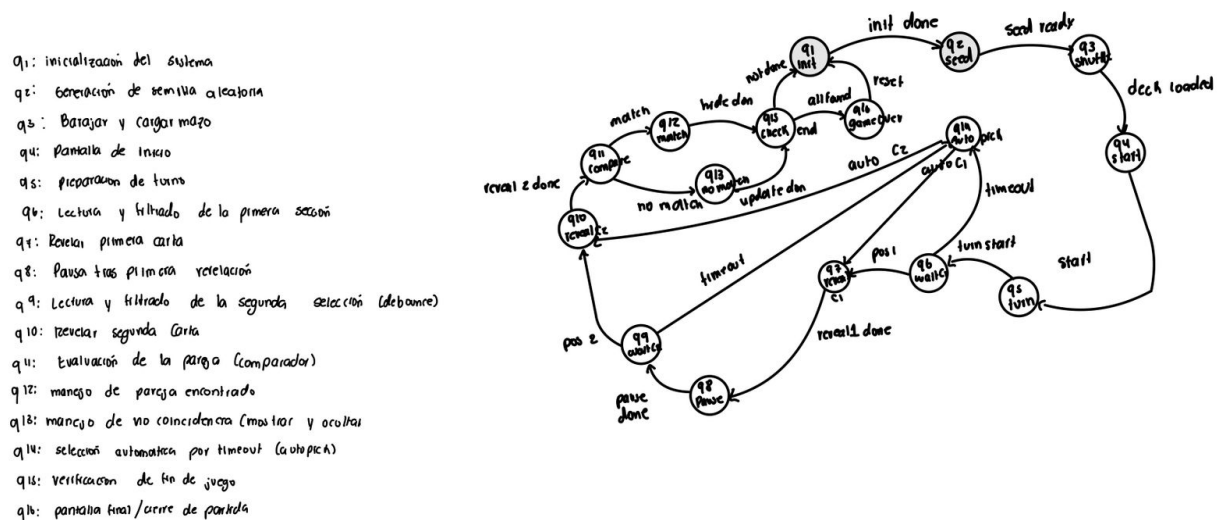


Figura 2: FSM seleccionada para la implementación final.